

# 作业 HW5 实验报告

姓名：庄子硕 学号：2350998 日期：2024 年 12 月 17 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

## 1. 涉及数据结构和相关背景

该题目要求理解和使用各种数据结构来解决不同的编程问题。包括：

- **子序列问题**：通过排序和前缀和数组（prefix sum）来优化暴力搜索，降低时间复杂度。
- **二叉查找树（BST）**：用于高效存储和查找整数集合，支持插入、删除、查找最小值、最大值等基本操作。
- **最少交换排序**：通过环结构的图理论方法解决最少交换次数问题。
- **家谱树**：构建家谱树并支持各种查询，如父母、子女、兄弟姐妹、祖先和后代关系的判断。

## 2. 实验内容

### 5.1 和有限的最长子序列

#### 5.1.1 问题描述

给你一个长度为  $n$  的整数数组 `nums` 和一个长度为  $m$  的整数数组 `queries`，返回一个长度为  $m$  的数组 `answer`，其中 `answer[i]` 是 `nums` 中元素之和小于等于 `queries[i]` 的子序列的最大长度。子序列是由一个数组删除某些元素（也可以不删除）但不改变元素顺序得到的一个数组。

#### 5.1.2 基本要求

第一行包括两个整数  $n$  和  $m$ ，分别表示数组 `nums` 和 `queries` 的长度  
第二行包括  $n$  个整数，为数组 `nums` 中元素  
第三行包含  $m$  个整数，为数组 `queries` 中元素

#### 5.1.3 数据结构设计

使用了排序和前缀和数组，通过二分查找来找到符合条件的最大子序列。

```
int nums[Seq_Maxlen];
int queries[Seq_Maxlen];
int answers[Seq_Maxlen];
```

#### 5.1.4 功能说明（函数、类）

通过预处理的前缀和数组和二分查找来求解每个查询的答案。通过优化算法避免暴力模拟的高时间复杂度。

```
int main()
```

```

{
    int n, m;
    cin >> n >> m;
    int nums[Seq_Maxlen];
    int queries[Seq_Maxlen];
    int answers[Seq_Maxlen];
    for (int i = 0; i < n; i++)
        cin >> nums[i];
    for (int i = 0; i < m; i++)
        cin >> queries[i];
    sort(nums, nums+n);
    int sum[Seq_Maxlen] = { 0 };
    for (int i = 0; i < n; i++)
        sum[i + 1] = sum[i] + nums[i];
    for (int i = 0; i < m; i++)
        answers[i] = (upper_bound(sum+1, sum +1+ n, queries[i])-(sum+1));
    for (int i = 0; i < m; i++)
        cout << answers[i] << ' ';
    return 0;
}

```

### 5.1.5 调试分析（遇到的问题和解决方法）

问题 1：模拟法时间复杂度过高

解决方法 1：先进行分析，找出答案的形式，再求解

### 5.1.6 总结和体会

这道题涉及到子序列的求解，需要对数组的元素进行排序，并利用前缀和来优化查询的效率。通过 `upper_bound` 查找满足条件的最大子序列长度，可以大大降低时间复杂度。解决过程中，我意识到暴力模拟虽然简单，但效率低下，在数据量较大时非常容易超时。因此，通过合理的排序与前缀和等技巧，我们能够有效地减少重复计算，提高程序的运行效率。总的来说，这道题让我深刻认识到算法优化的重要性，尤其是在面对大规模数据时，如何合理利用数据结构来提升性能是非常关键的。

## 5.2 题目二

### 5.2.1 问题描述

二叉排序树 BST（二叉查找树）是一种动态查找表，基本操作集包括：创建、查找、插入、删除、查找最大值、查找最小值等。

本题实现一个维护整数集合（允许有重复关键字）的 BST，并具有以下功能：1. 插入一个整数 2. 删除一个整数 3. 查询某个整数有多少个 4. 查询最小值 5. 查询某个数字的前驱（集合中比该数字小的最大值）。

### 5.2.2 基本要求

输入: 第 1 行一个整数 n, 表示操作的个数;

接下来 n 行, 每行一个操作, 第一个数字 op 表示操作种类:

若 op=1, 后面跟着一个整数 x, 表示插入数字 x

若 op=2, 后面跟着一个整数 x, 表示删除数字 x (若存在则删除, 否则输出 None, 若有多  
个则只删除一个),

若 op=3, 后面跟着一个整数 x, 输出数字 x 在集合中有多少个 (若 x 不在集合中则输出 0)

若 op=4, 输出集合中的最小值 (保证集合非空)

若 op=5, 后面跟着一个整数 x, 输出 x 的前驱 (若不存在前驱则输出 None, x 不一定在集  
合中)

输出: 一个操作输出 1 行 (除了插入操作没有输出)

### 5.2.3 数据结构设计

定义了一个 Node 结构体, 每个节点保存值和重复次数, 以及指向左右子树的指针。BST 类提供了对这些节点的插入、删除、查找等功能。

```
struct Node {
    int value;
    int repetition;
    Node* lnode = nullptr;
    Node* rnode = nullptr;
};

void Delete_Tree(Node*& nod1)
{
    if (nod1->lnode)Delete_Tree(nod1->lnode);
    if (nod1->rnode)Delete_Tree(nod1->rnode);
    delete nod1;
}

class BST {
private:
public:
    Node* Root;
    BST() {
        Root = new Node;
        Root->repetition = 0; //表示空表
    }
    ~BST() {
        Delete_Tree(Root);
    }
}
```

### 5.2.4 功能说明 (函数、类)

提供了多个方法, 如 Insert、Delete、Search 等, 分别处理节点的插入、删除、查找等操作。特别注意删除操作需要处理三种情况: 叶子节点、只有左子树、只有右子树。

```

void Insert(const int& ins)
{
    Node* Cur_Node = Root;

    if (Root->repetition == 0)
    {
        Root->value = ins;
        Root->repetition = 1;
        return;
    }

    while (1)
    {
        if (ins < Cur_Node->value)
        {
            if (Cur_Node->lnode != nullptr)
                Cur_Node = Cur_Node->lnode;
            else
            {
                Cur_Node->lnode = new Node;
                Cur_Node->lnode->value = ins;
                Cur_Node->lnode->repetition = 1;
                break;
            }
        }
        else if (ins > Cur_Node->value)
        {
            if (Cur_Node->rnode != nullptr)
                Cur_Node = Cur_Node->rnode;
            else
            {
                Cur_Node->rnode = new Node;
                Cur_Node->rnode->value = ins;
                Cur_Node->rnode->repetition = 1;
                break;
            }
        }
        else
        {
            Cur_Node->repetition += 1;
            break;
        }
    }
}

```

```

int Search(const int& ins, Node*& u_search, Node*& u_want_parent)
{
    Node* Cur_Node = Root;
    u_want_parent = nullptr;
    while (1)
    {
        if (ins < Cur_Node->value)
        {
            if (Cur_Node->lnode != nullptr)
            {
                u_want_parent = Cur_Node;
                Cur_Node = Cur_Node->lnode;
            }
            else
                return 0;
        }
        else if (ins > Cur_Node->value)
        {
            if (Cur_Node->rnode != nullptr)
            {
                u_want_parent = Cur_Node;
                Cur_Node = Cur_Node->rnode;
            }
            else
                return 0;
        }
        else
        {
            u_search = Cur_Node;
            return 1;
        }
    }
}

int Search_repetition(const int& search)
{
    Node* u_want;
    Node* u_want_parent;
    if (!Search(search, u_want, u_want_parent)) return 0; //没有找到
    return u_want->repetition;
}

Node* find_forw(const int& search)
{
    Node* Cur_Node = Root;
    Node* forw_Node = nullptr;

```

```

while (1)
{
    if (Cur_Node == nullptr)
        return forw_Node;

    if (Cur_Node->value < search)
    {
        forw_Node = Cur_Node;
        Cur_Node = Cur_Node->rnode;
    }
    else if (Cur_Node->value > search)
    {
        Cur_Node = Cur_Node->lnode;
    }
    else
    {
        forw_Node = Cur_Node->lnode == nullptr ? forw_Node :
Find_max(Cur_Node->lnode);
        return forw_Node;
    }
}

Node* Find_max(Node* root)
{
    Node* Cur_Node = root;
    while (Cur_Node->rnode != nullptr)
        Cur_Node = Cur_Node->rnode;

    return Cur_Node;
}

Node* Find_min(Node* root, Node*&parent)
{
    Node* Cur_Node = root;
    parent = nullptr;
    while (Cur_Node->lnode != nullptr)
    {
        parent = Cur_Node;
        Cur_Node = Cur_Node->lnode;
    }

    return Cur_Node;
}

int Delete(const int& dele)

```

```

{
    Node* u_want;
    Node* u_want_parent;
    Node* parent=NULLPTR;
    if (!Search(dele, u_want, u_want_parent))return 0;//没有找到

    if (u_want->repetition > 1)u_want->repetition -= 1;
    else
    {
        if (u_want->lnode == NULLPTR && u_want->rnode == NULLPTR)
        {
            if (u_want_parent == NULLPTR)
                Root->repetition = 0;
            else if (u_want_parent->lnode == u_want)u_want_parent->lnode =
NULLPTR;

            else if (u_want_parent->rnode == u_want)u_want_parent->rnode =
NULLPTR;

            if(u_want!=Root)delete u_want;
        }
        else if (u_want->rnode == NULLPTR)
        {
            if (u_want_parent == NULLPTR)
            {
                Node* mid = Root->lnode;
                delete Root;
                Root = mid;
                return 1;
            }

            else if (u_want_parent->lnode == u_want)u_want_parent->lnode =
u_want->lnode;

            else if (u_want_parent->rnode == u_want)u_want_parent->rnode =
u_want->lnode;

            if (u_want != Root)delete u_want;
        }
        else
        {
            Node* right_min = Find_min(u_want->rnode, parent);
            if (parent)
                parent->lnode = right_min->rnode;

            right_min->lnode = u_want->lnode;

```

```

        if (right_min != u_want->rnode)right_min->rnode = u_want->rnode;

        if (u_want_parent == nullptr)
        {
            delete Root;
            Root = right_min;
            return 1;
        }
        else if (u_want_parent->lnode == u_want)u_want_parent->lnode =
right_min;

        else if (u_want_parent->rnode == u_want)u_want_parent->rnode =
right_min;

        delete u_want;
    }
}

return 1;//成功删除
}
};

```

### 5.2.5 调试分析（遇到的问题和解决方法）

问题 1: RE 频繁

解决方案 1: 在调试过程中，主要解决了树的节点操作（如删除节点）中可能出现的空指针问题。

### 5.2.6 总结和体会

这道题让我深入理解了二叉排序树的基本操作，尤其是如何在树中进行插入和删除操作。通过递归与迭代方法实现树的各类操作，使我更加熟悉了树形数据结构的应用。

一个难点是在删除节点时，需要处理不同的情况（如叶子节点、只有一个子节点的节点、两个子节点的节点）。这个过程中遇到了一些挑战，尤其是在删除节点时的父节点链接调整。通过细致地分析与调试，最终解决了问题，并理解了二叉树结构的深层次细节。

总体来说，这道题让我对树形结构的操作有了更加深刻的理解，也锻炼了我的问题分析能力和调试能力。

## 5.3 题目三

### 5.3.1 问题描述

期末考试，监考老师粗心拿错了座位表，学生已经落座，现在需要按正确的座位表给学生重新排座。假设一次交换你可以选择两个学生并让他们交换位置，给你原来错误的座位表和正确的座位表，问给学生重新排座需要最少的交换次数。



### 5.3.2 基本要求

输入描述：两个  $n*m$  的字符串数组，表示错误和正确的座位表 `old_chart` 和 `new_chart`，`old_chart[i][j]` 为原来坐在第  $i$  行第  $j$  列的学生名字，对于 100% 的数据， $1 \leq n, m \leq 200$ ；人名为仅由小写英文字母组成的字符串，长度不大于 5

输出描述：一个整数，表示最少交换次数

### 5.3.3 数据结构设计

使用了map来记录学生名字到座位的映射关系。

```
map<string, int> ocm;
```

### 5.3.4 功能说明（函数、类）

通过查找交换环来减少交换次数。首先构建原始座位表到目标座位表的映射，然后通过检测并合并环来减少交换次数。

```
class Solution {
public:
    int solve(std::vector<vector<std::string>>& old_chart,
              std::vector<vector<std::string>>& new_chart) {
        // 这里填写你的代码
        int n = old_chart.size();
        int m = old_chart[0].size();

        int nodes = n * m;
        int rings = 0;

        //遍历表寻找交换环:一定有环，但个数不确定
        map<string, int> ocm;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                ocm.insert({ old_chart[i][j], i * m + j });
            }
        }

        vector<int> visited(n * m, 0);

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++)
            {
                int start_pos = i * m + j;
                string want_find = old_chart[i][j];
                if(visited[(ocm.find(want_find))>>second] != -1)
                    while (1)
                    {
                        //old_chart中对应位置
```

```

        int ncp=(ocm.find(want_find))->second;

        //同位置的newchart
        string nc = new_chart[ncp / m][ncp % m];

        //old_chart中这个new中字符的位置
        int ocp = (ocm.find(nc))->second;

        visited[ncp] = -1;//表示已经被访问过

        if (ocp == start_pos)//找到环了
        {
            rings++;
            break;
        }

        //该位置的oldchart
        want_find = old_chart[ocp / m][ocp % m];
    }
}

return nodes - rings;
};

```

### 5.3.5 调试分析（遇到的问题和解决方法）

问题 1：用 ocm 记录状态会出现错误

解决方案 1：设置专门的 visited 数组记录访问状态

问题 2：暴力法复杂度过高

解决方案 2：先分析，发现交换环模型

### 5.3.6 总结和体会

问题的主要挑战是如何识别并处理交换环，保证每个学生最终都能到达正确位置。

这道题本质上是一个关于排列的题目，涉及到如何通过交换操作将一个错误的座位表转换为正确的座位表。在实现过程中，我通过将每个座位的学生视为一个节点，研究了如何通过环的概念来优化交换次数。每一个环都可以通过最少的交换次数调整，因此最终的最少交换次数可以通过总节点数减去环的个数来得到。

这道题让我体会到在面对排列类问题时，如何从结构上拆解问题，识别出其中的规律，尤其是交换的最小化问题，如何通过图论的角度来简化问题。这个思路不仅限于座位表问题，也可以扩展到更多涉及排列的场景中。

## 5.3 题目三

### 5.4.1 问题描述

人类学教授对居住在孤岛上的人们及其历史感兴趣。他收集了他们的家谱进行一些人类学实验。为了实验，他需要用电脑来处理家谱。为此，他把它们翻译成文本文件。以下是表示家谱的文本文件的示例。

```
John
  Robert
    Frank
    Andrew
  Nancy
    David
```

每行包含一个人的名字。第一行的名字是这个家谱中最古老的祖先。家谱只包含该祖先的后代。他们的丈夫或妻子没有在家庭树上显示。一个人的孩子比父母缩进一个空格。例如，Robert 和 Nancy 是 John 的孩子，Frank 和 Andrew 是 Robert 的孩子。David 比 Robert 缩进了一个空格，但他不是 Robert 的孩子，而是 Nancy 的孩子。为了用这一方法表示一个家谱，教授将一些人从家谱中排除，使得家谱中没有人有两个父母。

为了实验，教授还收集了家属的文件，并提取了每个家谱中关于两人关系的陈述语句。以下是关于上述家庭陈述语句的一些例子。

```
John is the parent of Robert.
Robert is a sibling of Nancy.
David is a descendant of Robert.
```

对于实验，他需要判断每个陈述语句是否正确。例如，上面的前两个语句是正确的，最后一个语句是错误的。但是这个任务是十分无聊的，他想通过电脑程序来判断。

需要支持的查询有以下几种：

```
X is a child of Y.
X is the parent of Y.
X is a sibling of Y.
X is a descendant of Y.
X is an ancestor of Y.
```

注意：一个人的祖先、后代、兄弟可以是自己，但父母、孩子则不行。

### 5.4.2 基本要求

输入：输入包含若干组测试用例，每个测试用例由一个家谱和一个陈述句集合组成。每个测试用例的第一行给出两个整数  $n, m$  ( $0 < n, m < 1000$ )，分别表示家谱中的名字和陈述语句的数目。

接下来输入的每行少于 70 个字符。名字的字符串仅由不超过 20 个字母字符组成。在家谱的第一行中给出名字前没有前导空格，在家谱中其他的名字至少缩进一个空格，表示是第一行给出的那个名字（家族的最早祖先）的后代。若家谱中一个名字缩进  $k$  个空格，则下一行最多缩进  $k+1$  个空格。

本题假定，除了最早的祖先外，在家谱中，每个人都有其父母。在同一个家谱中同样的名字不会出现两次。家谱的每一行结束的时候无多余的空格。

再接下来每句陈述句占一行。在家谱中没有出现的名字不会出现在陈述句中。陈述句中连续的单词被一个空格分开。每句陈述句在行首和行尾没有多余的空格。用两个 0 表示输入的开始。

输出：对于测试用例中的每句陈述语句，程序输出一行，给出 True 或 False。  
每个测试用例后要给出一个空行，就算只有一个测试用例。

### 5.4.3 数据结构设计

使用了 Node 结构体来表示家谱中的每个成员，树的根节点为最早的祖先。每个节点存储该成员的父节点和所有子节点。

```
long long get_hash(const string& s1)
{
    long long code=0;
    for (int i = 0; i < s1.size(); i++)
    {
        code += (i + 1) *(s1[i]%10);
    }
    return code;
}
```

```
struct Node {
    Node* parent;
    vector<Node*> children;
    string name;
};
```

```
map<long long, vector<Node*>>hash_map;
```

### 5.4.4功能说明（函数、类）

提供了多种关系查询功能，包括判断父母、子女、祖先、后代、兄弟姐妹等关系。通过遍历树来判断关系。

```
int main()
{
    while (1)
    {
        int m, n;
        cin >> n >> m;

        //两个0表示结束
        if (!n && !m)break;

        //最先祖先的前一个结点
        Node* Before_Root = new Node;
        Before_Root->parent = nullptr;

        Node* CurParent = Before_Root;//这层Depth是-1
        int CurDepth = 0;
        // 读取家族树
```

```

for (int i = 0; i < n; i++)
{
    string name_line="";
    while(name_line.empty())
        getline(cin, name_line);

    //计算nameline前有几个空格，找到新行的深度
    int NewlineDepth = 0;
    for (int i = 0; i < name_line.size(); i++)
    {
        if (name_line[i] == ' ')NewlineDepth += 1;
        else
        {
            //更新为名字
            name_line=name_line.substr(i);
            break;
        }
    }
    //根据新行深度，找到新行的parent，并新增子节点

    Node* NewChild = new Node;
    // 如果新行深度小于现在行，由CurParent向上回溯，找到新行的parent并为他新增
子节点

    if (NewlineDepth < CurDepth)
    {
        while (NewlineDepth < CurDepth)
        {
            CurParent = CurParent->parent;
            CurDepth--;
        }
    }
    // 如果新行深度等于现在行，CurParent新增子结点
    else if (NewlineDepth == CurDepth)
    {
        ;
    }
    // 如果新行深度大于现在行，CurParent更新为现在行，为CurParent新增子节点
    else
    {
        CurParent = CurParent->children[CurParent->children.size() - 1];
        CurDepth++;
    }

    NewChild->name = name_line;

```

```

NewChild->parent = CurParent;

CurParent->children.push_back(NewChild);

// 增加子节点时, 计算子节点的hash值, 并把hash值同结点指针存入map
long long hash_code = get_hash(NewChild->name);
if (hash_map.find(hash_code) == hash_map.end())//没找到
{
    vector<Node*> newcode;
    newcode.push_back(NewChild);
    hash_map.insert({ hash_code, newcode });
}
else
    hash_map.find(hash_code)->second.push_back(NewChild);
}

// 读取陈述
// 只关心1、4、6项
for (int i = 0; i < m; i++)
{
    string Name1, item0, item1, relation, item2, Name2;
    cin >> Name1 >> item0 >> item1 >> relation >> item2 >> Name2;

    //Name2末尾有一个.
    Name2 = Name2.substr(0, Name2.size() - 1);

    Node* NodeName1 = nullptr, * NodeName2 = nullptr;
    long long hash4Name1, hash4Name2;
    hash4Name1 = get_hash(Name1);
    hash4Name2 = get_hash(Name2);

    // 查找Name对应结点
    for (Node*& node : (hash_map.find(hash4Name1)->second))
    {
        if (node->name == Name1)
        {
            NodeName1 = node;
            break;
        }
    }

    for (Node*& node : (hash_map.find(hash4Name2)->second))
    {
        if (node->name == Name2)

```

```

        {
            NodeName2 = node;
            break;
        }
    }

    if (relation == "child")
    {
        bool is_child = false;
        for (Node*& node : NodeName2->children)
        {
            if (node->name == NodeName1->name)
            {
                is_child = true;
                break;
            }
        }

        if (is_child) cout << "True" << endl;
        else cout << "False" << endl;
    }
    else if (relation == "ancestor")
    {
        vector<string> ancestors;
        ancestors.push_back(NodeName2->name);

        Node* CurNode=NodeName2;
        while (CurNode->parent->name != Before_Root->name)
        {
            CurNode = CurNode->parent;
            ancestors.push_back(CurNode->name);
        }

        //检查是否在祖先列中
        bool is_ancestor = false;
        for (string& ancestor : ancestors)
        {
            if (ancestor == NodeName1->name)
            {
                is_ancestor = true;
                break;
            }
        }
    }

```

```

        if (is_ancestor)cout << "True" << endl;
        else cout << "False" << endl;
    }
else if (relation == "sibling")
{
    //检查是否在兄弟列中
    bool is_sibling = false;
    for (Node*& sibling : NodeName2->parent->children)
    {
        if (sibling->name == NodeName1->name)
        {
            is_sibling = true;
            break;
        }
    }

    if (is_sibling)cout << "True" << endl;
    else cout << "False" << endl;
}
else if (relation == "parent")
{
    bool is_parent = false;
    if (NodeName1->name == NodeName2->parent->name)is_parent = true;

    if (is_parent)cout << "True" << endl;
    else cout << "False" << endl;
}
else if (relation == "descendant")
{
    //name1是name2后代，那么name2是name1前代

    vector<string> ancestors_for_name1;
    ancestors_for_name1.push_back(NodeName1->name);

    Node* CurNode = NodeName1;
    while (CurNode->parent->name != Before_Root->name)
    {
        CurNode = CurNode->parent;
        ancestors_for_name1.push_back(CurNode->name);
    }

    //检查name2是否在name1祖先列中
    bool is_desecent = false;
    for (string& ancestor : ancestors_for_name1)

```



```

        {
            if (ancestor == NodeName2->name)
            {
                is_desecent = true;
                break;
            }
        }

        if (is_desecent) cout << "True" << endl;
        else cout << "False" << endl;
    }
}

//每个测试用例后给出一个空行
cout << endl;
hash_map.clear();
}

return 0;
}

```

#### 5.4.5 调试分析（遇到的问题和解决方法）

问题 1：时间复杂度过高

解决方案 1：用哈希表，注意每一项不要过长，但也不要每个 hash 值只对应一个元素

问题 2：多次问题时出现 bug

解决方案 2：没有清空 hash 表

#### 5.4.6 总结和体会

在树的构建和关系查询过程中，主要解决了如何高效地遍历树并避免重复计算的问题。这道题综合了树形结构、哈希表以及字符串匹配等多个知识点，涉及到如何高效地表示家谱结构，并基于这个结构处理多种查询操作。通过使用哈希表来快速定位结点，再结合深度优先搜索 (DFS) 来寻找父母、子女、兄弟等关系，我成功实现了各类查询操作。

在实现过程中，我特别注重了家谱的树形结构构建，如何通过深度信息将每个人与其父母、孩子正确连接，并确保查询操作的高效性。遇到的挑战主要是如何正确地管理树形结构中的关系，尤其是祖先与后代的判断，以及如何高效地处理多次查询。

### 3. 实验总结

在本次实验中，通过多种数据结构的应用，深入理解了如何用合适的算法和数据结构来解决实际问题。每个问题的解决都依赖于对数据结构的深入理解和算法的优化。在调试过程中，遇到的问题大多是如何高效处理大数据量和复杂关系查询的问题，经过优化后解决了时间复杂度过高的问题。本次实验加深了对数据结构（如二叉查找树、排序、图、树等）的理解，并提升了编程能力和解决实际问题的能力。